AUTOMATED THEOREM PROVING / 001

素数无穷性的构造性证明

对“任意自然数 n，存在素数 p > n”的形式化论证与计算验证

证明方法 **欧几里得反证法** 验证环境 **Python 3.12.14** 报告日期 **2026 年 10 月 25 日**

摘要 / ABSTRACT

本报告证明：对每个自然数 n，都存在素数 p 严格大于 n。论证采用欧几里得的有限清单构造——列出不超过 n 的全部素数，令 N 为它们的乘积加一，则 N 的任一素因子都落在清单之外，因而必大于 n。报告随后将论证逐步形式化，审计“N 的整除性”“N > 1”等疑点，并以 Python 对 n = 100 及多组边界值进行验证。

关键词 素数无穷性 · 欧几里得构造 · 反证法 · 计算验证

结论 $\forall\, n \in \mathbb{N}\ \exists p \in \mathbb{P}: p > n$

任意自然数都可被某个更大的素数超越。

n = 100 → **p = 101**

01 / 知识检索

欧几里得的经典论证

《几何原本》第九卷命题 20 说：素数比任何给定的素数集合都多。证明不靠“再找一个素数”，而是把任意有限清单当作已经穷尽全部素数，再构造一个清单无法解释的对象。

01 假设

设 $p_1, p_2, \ldots, p_k$ 是全部素数。清单有限，且可按任意顺序枚举。

02 构造

令 $N = p_1 p_2 \cdots p_k + 1$。N 比清单中的每个素数都大。

03 取素因子

N > 1，故 N 必有素因子 q。注意：q 未必等于 N。

04 导出矛盾

每个 p_i 都整除乘积、却不整除 N，所以 q 不在清单中。

原始命题 / EUCLID · ELEMENTS IX.20

“素数比任何指定的素数集合都多。”

证明只依赖三件事：清单有限、整除的线性性质、以及每个大于 1 的整数都有素因子。

02 / 形式化构建

从反证叙事到逐步推理

THEOREM 2.1 / 欧几里得

对任意自然数 n，存在素数 p，使得 $p > n$。

证明 固定任意 $n \in \mathbb{N}$，按 n 的大小分两种情形。以下每一步只使用前一列出的定义或已经确立的事实。

步骤	断言 / 构造	依据
P1	固定任意 $n \in \mathbb{N}$。	全称目标的引入规则
P2	若 $n < 2$，取 $p = 2$。	$2 \in \mathbb{P}$ 且 $2 > n$
P3	否则 $n \geq 2$。令 $S_n = \{p \in \mathbb{P} : p \leq n\} = \{p_1, \ldots, p_k\}$。	S_n 是有限集；$2 \in S_n$，故 $k \geq 1$
P4	构造 $N = p_1 p_2 \cdots p_k + 1$。	整数乘法与加法封闭；$N \geq 3 > 1$
P5	存在素数 q，使得 $q \mid N$。	大于 1 的整数必有素因子
P6	对每个 i，$N \equiv 1 \pmod{p_i}$，故 $p_i \nmid N$；于是 $q \neq p_i$。	若 $p_i \mid N$ 且 $p_i \mid$ 乘积，则 $p_i \mid 1$，矛盾
P7	$q \in \mathbb{P}$ 且 $q \notin S_n$，因此 $q > n$。	S_n 恰含所有不超过 n 的素数

两个情形的共同结论 无论 n 是否小于 2，构造都给出了一个严格大于 n 的素数，故全称命题成立。∎

> **为什么这不是循环论证？**
>
> 我们只假设 S_n 有限——它作为 $\{1, 2, \ldots, n\}$ 的子集当然有限；并没有假设全体素数有限。构造从任意上界 n 出发，产生一个落在 S_n 之外的素数，因此每一步都没有预先使用待证结论。

整除性的核心

$$N = 1 + p_1 p_2 \cdots p_k \Rightarrow N \equiv 1 \pmod{p_i} \Rightarrow p_i \nmid N$$

因此清单中的每个素数都在 N 上留下余数 1；这正是“新素数”无法留在清单里的原因。

> **构造性注记 / CONSTRUCTIVE, NOT EFFICIENT**
>
> 证明是构造性的：输入 n，理论上可枚举 S_n、计算 N、再分解 N 得到 q。但 N 随 n 指数增长，显式分解可能非常昂贵；该论证保证的是存在性，而不是计算效率。

03 / 自我反思

逐项审计证明的薄弱点

证明的大结构并不复杂，风险通常藏在被默认为显然的小事实里。下面把可能被质疑的环节逐项展开；其中“N 本身是否素数”是最值得纠正的表述。

S_n 有限 → $N > 1$ → $q \mid N$ → $q \notin S_n$ → $q > n$

审计清单

检查点	结论	说明
N 能被某个 p_i 整除吗？	✓ 不能	$N = 1 + (p_i$ 的倍数)，故 $N \equiv 1 \pmod{p_i}$。带余除法给出余数 1，不是 0。
N 一定大于 1 吗？	✓ 一定	$n \geq 2$ 时清单含 2，乘积 ≥ 2，因此 $N \geq 3$。$n < 2$ 的情形已单独取 $p = 2$。
N 本身一定是素数吗？	✓ 不一定	$2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 + 1 = 30031 = 59 \times 509$。证明只需 N 的某个素因子 q，不能把 N 直接当作素数。
是否预设了“全体素数有限”？	✓ 没有	只使用 $S_n = \{p \leq n\}$ 有限；这是有界集合，不是全体素数。
q 会不会仍等于某个 p_i？	✓ 不会	若 $q = p_i$，则 p_i 同时整除 N 与乘积，于是 $p_i \mid 1$，矛盾。
q 一定严格大于 n 吗？	✓ 一定	q 是素数且不在 S_n 中，而 S_n 已包含所有不超过 n 的素数，故 $q > n$。
n = 0 或 n = 1 怎么办？	✓ 单独处理	取 $p = 2$；2 是素数且 $2 > 0$、$2 > 1$，不进入构造分支。

审计结论 / PASS

整除性、N > 1、非循环性与边界情形均已闭合。唯一需要修正的表述是：不能说“N 是素数”，只能说“N 有一个素因子 q，且 q 在清单之外”。

04 / 代码验证

用可执行程序检验结论

程序直接执行存在性断言：从 n + 1 开始逐个检查，返回第一个素数。判定使用精确试除法，只需检查到 ⌊√c⌋；这与证明的初等风格一致。

输入 n = 100 → 输出 p = 101

101 是素数 · 101 > 100 · 间隙 = 1

回归测试

输入 n	输出 p	p − n	is_prime(p)	p > n
0	**2**	2	True	True
1	**2**	1	True	True
2	**3**	1	True	True
100	**101**	1	True	True
1 000	**1 009**	9	True	True
10 000	**10 007**	7	True	True
1 000 000	**1 000 003**	3	True	True

核心代码

```
def is_prime(c):
    if c < 2:
        return False
    d = 2
    while d * d <= c:
        if c % d == 0:
            return False
        d += 1
    return True

def next_prime(n):
    c = n + 1
    while not is_prime(c):
        c += 1
    return c
```

教学简化版：交付源码使用 6k ± 1 步进优化，判定逻辑等价。

实际运行 `$ python prime_greater_than.py 100`

```
n = 100   →   p = 101
is_prime(p) = True    p > n = True    ALL CHECKS PASSED
```

结语 数学证明给出了对所有自然数的保证；程序则在具体输入上复核了存在性与严格不等式。二者相互独立，又指向同一结论。

附：完整源码 `prime_greater_than.py` · 完整运行记录 `verification_output.txt`